

Architectural Style Justification

Lab 3: Component Modelling & Architectural Pattern Selection | Blood Bank Inventory & Emergency Donor Matcher System (Problem Statement #12)

Satvik Kalyanpur • PES1UG24CS617 • Section K

Architecture Selection: “We chose Microservices Architecture for the Blood Bank Inventory & Emergency Donor Matcher System.”

Architectural Choice

Microservices Architecture was selected over the Layered and Client-Server alternatives. The system decomposes cleanly into independent, loosely-coupled services — Emergency Request Manager, Inventory Ledger, Donor Matching & Alert Engine, Donor Registry, Notification Adapter, and Security & Audit Service — each owning a distinct responsibility and communicating over well-defined APIs.

Two Reasons

Reason 1 — Divergent load and consistency needs: The Inventory Ledger must guarantee strict atomic, transactional locking so that a single blood bag is never double-allocated, holding median latency ≤ 500 ms under 200 concurrent allocation requests (NFR-001). The Donor Matching & Alert Engine, by contrast, is a bursty, best-effort workflow gated by two external, potentially slow integrations (Geolocation Service, SMS Gateway) with a softer 30-second SLA (FR-001). Splitting these into separate services lets each be scaled and tuned to its own load profile instead of sharing resources and contention.

Reason 2 — Fault isolation of external dependencies: Donor alert broadcasting (Alternate Flow 7a) depends on third-party SMS Gateway and Geolocation Service APIs that can be slow, rate-limited, or unavailable. Isolating this logic inside the Donor Matching & Alert Engine and Notification Adapter services ensures that a failure or latency spike there cannot block or slow the safety-critical emergency reservation transaction in the Inventory Ledger, keeping the core allocation path responsive and independently deployable/testable.

Security Advantage

Donor PII protection and audit logging (NFR-002) are concentrated inside one dedicated Security & Audit microservice, exposed to every other service through a single `ISecurityAudit` interface. Encryption at rest (AES-256), encryption in transit (TLS 1.3), role-based access control, and the immutable audit entry required for every access to donor contact details are therefore enforced at one hardened boundary rather than re-implemented (and potentially mis-configured) inside each service that touches donor data — shrinking the attack surface and making the mandated audit trail complete by construction.

Performance Benefit

Because the Inventory Ledger is its own microservice, it can be independently scaled and optimised (dedicated database, in-memory locking) to meet the strict ≤ 500 ms median-latency target under 200 concurrent allocation requests, without contention from unrelated, higher-latency workloads such as donor geolocation lookups or SMS dispatch — which would otherwise share resources and queue behind ledger transactions inside a single monolithic layer.